

Lecture 4: advanced synchronization primitives

monitor, synchronized, completion service, thundering herd problem, wait morphing, countdown latch, semaphore, read-write lock, thread local, replication, snapshotting

Alexander Filatov
filatovaur@gmail.com

<https://github.com/Svazars/parallel-programming/blob/main/slides/pdf/l4.pdf>

In previous episodes

- We study communication and coordination of threads in pre-emptive multitasking OS
- Execution speed is not under our control
- Decoupling tasks – Runnable, Callable, Future
 - from particular threads – ThreadFactory
 - from submission protocol – Executor
 - from scheduling policy – ExecutorService
- Tools for thread coordination:
 - Thread.join, Lock, Condition, ExponentialBackoff, Future.get
- Concurrency pitfalls
 - wait-for graph, deadlock, thread-starvation deadlock, resource deadlock
 - fairness, priority inversion, starvation
 - lost signal, spurious wakeup, predicate invalidation

Lecture plan

- 1 Monitor
 - API
 - Wait Morphing
- 2 Broadcast
 - CountdownLatch
 - Thundering herd problem
- 3 Group-level concurrency
 - Semaphore
 - ReadWriteLock
- 4 ThreadLocal
 - API
 - Replication
 - Snapshotting
- 5 Thread-safe counters
- 6 Summary

Mutex: popular use-cases

```
interface Lock {  
    void lock();  
    void unlock();  
}
```


Question time

Question: OOP, modularity, code reuse – which mutex property helps to support it?



Mutex: popular use-cases

```
interface Lock {  
    void lock();  
    void unlock();  
}
```

- Reentrant

Question time

Question: why do we develop multithreaded software?



Mutex: popular use-cases

```
interface Lock {  
    void lock();  
    void unlock();  
}
```

- Reentrant
- Throughput-oriented

Question time

Question: fair or unfair mutex helps to achieve better throughput of system-as-a-whole?



Mutex: popular use-cases

```
interface Lock {  
    void lock();  
    void unlock();  
}
```

- Reentrant
- Unfair

Question time

Question: mutex solves problem of thread **contention**. What should we use to simplify thread **coordination**?



Mutex: popular use-cases

```
interface Lock {  
    void lock();  
    void unlock();  
    Condition newCondition();  
}
```

- Reentrant
- Unfair

```
interface Condition {  
    void await();  
    void signal();  
    void signalAll();  
}
```


Question time

Question: how many Condition Variables do you need for the most use cases we studied so far?



Mutex: popular use-cases

```
interface Lock {  
    void lock();  
    void unlock();  
    Condition condition();  
}
```

- Reentrant
- Unfair
- Single condition

```
interface Condition {  
    void await();  
    void signal();  
    void signalAll();  
}
```

Mutex: popular use-cases

```
interface PopularLock {  
    void lock();  
    void unlock();  
  
    void await();  
    void signal();  
    void signalAll();  
}
```

- Reentrant
- Unfair
- Single condition

Question time

Question: control flow and mutex – any pitfalls?



Mutex: popular use-cases

```
interface PopularLock {  
    void lock();  
    void unlock();  
  
    void await();  
    void signal();  
    void signalAll();  
}
```

- Reentrant
- Unfair
- Single condition
- Structured locking: lock try-finally unlock

Question time

Question: how to strictly enforce structured locking on source code level?



Mutex: popular use-cases

```
interface PopularLock {  
    // lock-unlock provided by language keyword  
    void await();  
    void signal();  
    void signalAll();  
}
```

- Reentrant
- Unfair
- Single condition
- Structured locking: language support

Mutex: popular use-cases

```
interface PopularLock {  
    // lock-unlock provided by language keyword  
    void await();      // <-- this  
    void signal();     //           could  
    void signalAll();  //           clash  
}
```

- Reentrant
- Unfair
- Single condition
- Structured locking: language support

Mutex: popular use-cases

```
interface PopularLock {  
    // lock-unlock provided by language keyword  
    void wait();  
    void notify();  
    void notifyAll();  
}
```

- Reentrant
- Unfair
- Single condition
- Structured locking: language support

Lecture plan

- 1 Monitor
 - API
 - Wait Morphing
- 2 Broadcast
 - CountdownLatch
 - Thundering herd problem
- 3 Group-level concurrency
 - Semaphore
 - ReadWriteLock
- 4 ThreadLocal
 - API
 - Replication
 - Snapshotting
- 5 Thread-safe counters
- 6 Summary

Monitor

```
class Monitor { void wait();  
                void notify();  
                void notifyAll();  
}
```

- Reentrant, Unfair
- synchronized for critical section (always structured)

```
void main() {  
    var monitor = ...;  
    synchronized (monitor) {  
        foo();  
        if (bar()) { monitor.wait(); }  
        else      { monitor.notify(); }  
    }  
}
```

Question time

Question: in which Java package Monitor class is declared?



Built-in Java monitor

```
java.lang.Object {  
    void wait();  
    void notify();  
    void notifyAll();  
}
```

- Reentrant, Unfair
- synchronized for critical section (always structured)
- **Every** Java object has associated monitor

<https://docs.oracle.com/en/java/javase/11/docs/api/java.base/java/lang/Object.html>

Built-in Java monitor

```
java.lang.Object {  
    void wait();  
    void notify();  
    void notifyAll();  
}
```

- Reentrant, Unfair
- synchronized for critical section (always structured)
- **Every** Java object has associated monitor

<https://docs.oracle.com/en/java/javase/11/docs/api/java.base/java/lang/Object.html>

It **does not** mean you allocate monitor for every object, though

- "Thin locks: featherweight synchronization for Java" by Bacon et al. 1998¹

¹<https://dl.acm.org/doi/abs/10.1145/277652.277734>

Disclaimer

Concurrent programming is rapidly evolving branch of Computer Science.

Disclaimer

Concurrent programming is rapidly evolving branch of Computer Science.

There are many designs for key synchronization primitives:

- "Monitor classification" by Buhr et al²
- "Semaphores in Plan 9" by Mullender and Cox³
- "Futexes are tricky" by Drepper⁴

² <https://dl.acm.org/doi/10.1145/214037.214100>

³ <https://swtch.com/semaphore.pdf>

⁴ <https://dept-info.labri.fr/~denis/Enseignement/2008-IR/Articles/01-futex.pdf>

Disclaimer

Concurrent programming is rapidly evolving branch of Computer Science.

There are many designs for key synchronization primitives:

- "Monitor classification" by Buhr et al²
- "Semaphores in Plan 9" by Mullender and Cox³
- "Futexes are tricky" by Drepper⁴

Today we will stick to the Java programming language (built-in monitors) and `java.util.concurrent` package.

² <https://dl.acm.org/doi/10.1145/214037.214100>

³ <https://swtch.com/semaphore.pdf>

⁴ <https://dept-info.labri.fr/~denis/Enseignement/2008-IR/Articles/01-futex.pdf>

Disclaimer

Concurrent programming is rapidly evolving branch of Computer Science.

There are many designs for key synchronization primitives:

- "Monitor classification" by Buhr et al²
- "Semaphores in Plan 9" by Mullender and Cox³
- "Futexes are tricky" by Drepper⁴

Today we will stick to the Java programming language (built-in monitors) and `java.util.concurrent` package.

It is enough to illustrate design space, trade-offs, common correctness problems.

² <https://dl.acm.org/doi/10.1145/214037.214100>

³ <https://swtch.com/semaphore.pdf>

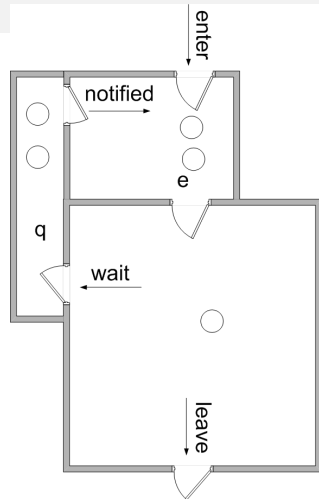
⁴ <https://dept-info.labri.fr/~denis/Enseignement/2008-IR/Articles/01-futex.pdf>

Java monitor

Design

Effectively ReentrantLock + single Condition

- MonitorEnter^{5,6}
- MonitorExit
- wait
- notify
- notifyAll



⁵ <https://docs.oracle.com/javase/specs/jvms/se11/html/jvms-6.html#jvms-6.5.monitorenter>

⁶ <https://docs.oracle.com/javase/specs/jvms/se11/html/jvms-3.html#jvms-3.14>

Toy problem: CompletionService

<https://docs.oracle.com/en/java/javase/11/docs/api/java.base/java/util/concurrent/CompletionService.html>

```
interface CompletionService<V> {  
    Future<V> submit(Callable<V> task);  
    Future<V> take();  
}
```

- take retrieves the Future representing the next completed task, blocks if no available
- thread-safe dequeue
 - producers submit tasks
 - consumers process completed tasks

CompletionService via built-in monitors

```
class MyCompletionService<V> implements CompletionService<V> {
    ExecutorService service; Queue<V> q = new LinkedList<>();
    MyCompletionService(ExecutorService service) { this.service = service; }
    Future<V> submit(Callable<V> task) {
        return service.submit(() -> {
            V result = task.call();
            synchronized (q) { q.add(result); q.notify(); }
        });}
    Future<V> take() {
        synchronized (q) {
            while (q.isEmpty()) { q.wait(); }
            return new AlreadyCompletedFuture<>(q.remove());
        }}}
```

Monitoring at home

<https://github.com/Svazars/parallel-programming/tree/main/hw/block1/4.1>

Homework

Task 4.1

- *Find bugs in small programs that use built-in monitors*

Java monitor: for every object

Every Java object have associated built-in monitor.

```
Object x = new Object();  
synchronized(x) {  
    x.wait();  
    x.notifyAll();  
}
```

- Special keyword: `synchronized` (no `.lock/unlock`, no `try-finally`, always structured)
- `Condition` and `ReentrantLock` are Java objects and they have built-in monitor
- Library-level synchronization and language-level synchronization are **independent**, do not mix `notify/signal` and `wait/await`.

Java monitor: spot-a-problem №1

```
Lock l = ...;  
synchronized (l) {  
    foo();  
}
```


Java monitor: spot-a-problem №1

```
Lock l = ...;  
synchronized (l) {  
    foo();  
}
```

synchronized -> monitorenter/monitorexit

Java monitor: spot-a-problem №1

```
Lock l = ...;  
synchronized (l) {  
    foo();  
}
```

synchronized -> monitorenter/monitorexit
monitorenter != Lock.lock

Java monitor: spot-a-problem №2

```
Lock l = ..;  
l.lock();  
try {  
    while (!bar()) {  
        l.wait();  
    }  
    baz();  
} finally {  
    l.unlock();  
}
```

Java monitor: spot-a-problem №2

```
Lock l = ..;  
l.lock();  
try {  
    while (!bar()) {  
        l.wait();  
    }  
    baz();  
} finally {  
    l.unlock();  
}
```

wait -> built-in monitor associated with l, not some Condition

Java monitor: spot-a-problem №3

```
Lock l = ..;  
Condition c = l.newCondition();  
...  
l.lock();  
try {  
    while (!bar()) {  
        c.wait();  
    }  
    baz();  
} finally {  
    l.unlock();  
}
```

Java monitor: spot-a-problem №3

```
Lock l = ..;  
Condition c = l.newCondition();  
...  
l.lock();  
try {  
    while (!bar()) {  
        c.wait();  
    }  
    baz();  
} finally {  
    l.unlock();  
}
```

wait -> built-in monitor associated with c, not Condition.await

Java monitor: for every method

Every Java method could be marked as synchronized.

```
public void foo() { doStuff(); }  
public synchronized void bar() { doStuffUnderMutex(); }
```

It synchronizes on this (instance methods) or Class<?> (static methods):

```
public synchronized void fooSignature() { doStuff(); }  
public void fooInternal() {  
    synchronized(this) {  
        doStuff();  
    }  
}
```

Java monitor: spot-a-problem №4

```
class MyClass {  
    private final Object guard = new Object();  
    public void foo() { synchronized(guard) { bar(); } }  
    public synchronized void bar() { }  
}
```


Java monitor: spot-a-problem №4

```
class MyClass {  
    private final Object guard = new Object();  
    public void foo() { synchronized(guard) { bar(); } }  
    public synchronized void bar() { }  
}  
  
class OhMyClass extends MyClass {  
    public synchronized void baz() {  
        foo();  
    }  
}
```

Java monitor: spot-a-problem №4

```
class MyClass {  
    private final Object guard = new Object();  
    public void foo() { synchronized(guard) { bar(); } }  
    public synchronized void bar() { }  
}  
  
class OhMyClass extends MyClass {  
    public synchronized void baz() {  
        foo();  
    }  
}
```

Possible deadlock:

- Thread A calls foo: lock(guard) -> lock(this)
- Thread B calls baz: lock(this) -> lock(guard)

Question time

Question: Is it good or bad to mark methods as thread-safe (synchronized on publicly available signature level)?



Java monitor: for every method

Every Java method could be marked as synchronized.

```
public void foo() { doStuff(); }  
public synchronized void bar() { doStuffunderMutex(); }
```

Inheritance or `synchronized(myAwesomeClassInstance)`

- may violate your locking policy
- may cause a deadlock

It is common pattern to use `private final Object lock` and do not expose it as public API.

Java monitor

Common pitfalls

Correctness/safety:

- Blocking `MonitorEnter`, `wait` – deadlock, locking order
- Reentrant
- Visibility/consistency – `synchronized` looks like "atomic transaction"
- Lost signal
- Predicate invalidation
- Spurious wakeup

Fairness/starvation:

- Admission policy for `MonitorEnter`/`MonitorExit` is not specified
- Admission policy for `wait`/`notify` is not specified

Question time

Question: what is spurious wakeup for built-in Java monitor?



Homebrewed with love

<https://github.com/Svazars/parallel-programming/tree/main/hw/block1/4.2>

Homework

Task 4.2

- *Implement Monitor using ReentrantLock and Condition*
- **Hard:** *implement it without spurious wakeups*

Java monitor

Specific pitfalls

Problem: unintended mix of `java.util.concurrent` and built-in synchronization

- `wait` vs `await`
- `synchronized` vs `lock/unlock`

Solution: static analysis (FindBugs/IntelliJ IDEA)

Java monitor

Specific pitfalls

Problem: unintended mix of `java.util.concurrent` and built-in synchronization

- `wait` vs `await`
- `synchronized` vs `lock/unlock`

Solution: static analysis (FindBugs/IntelliJ IDEA)

Problem: unintended encapsulation breach

- `synchronized` methods inheritance
- `synchronized` on object outside of library method

Solution: `private final` guards

Java monitor

Rules of thumb

Mutual exclusion

- If you need "just mutex" – consider using encapsulated object + synchronized
 - Save your time with try-finally
 - Other stuff (built-ins have better monitoring and optimizer-friendly) is debatable

Java monitor

Rules of thumb

Mutual exclusion

- If you need "just mutex" – consider using encapsulated object + synchronized
 - Save your time with try-finally
 - Other stuff (built-ins have better monitoring and optimizer-friendly) is debatable
- Use Lock if you need "advanced" properties:
 - Fairness (remember, scheduling is counter-intuitive)
 - Non-structured locking (remember, exceptions are hard)
 - Polling via tryLock (remember, race conditions are tricky)

Java monitor

Rules of thumb

Mutual exclusion

- If you need "just mutex" – consider using encapsulated object + synchronized
 - Save your time with try-finally
 - Other stuff (built-ins have better monitoring and optimizer-friendly) is debatable
- Use Lock if you need "advanced" properties:
 - Fairness (remember, scheduling is counter-intuitive)
 - Non-structured locking (remember, exceptions are hard)
 - Polling via tryLock (remember, race conditions are tricky)

Signalling

Java monitor

Rules of thumb

Mutual exclusion

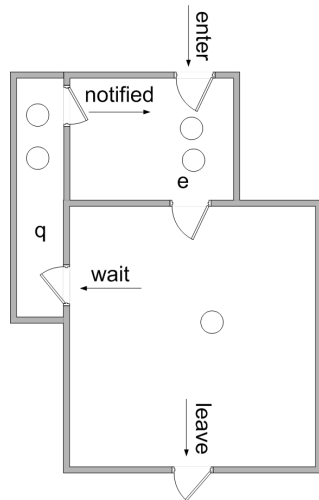
- If you need "just mutex" – consider using encapsulated object + synchronized
 - Save your time with try-finally
 - Other stuff (built-ins have better monitoring and optimizer-friendly) is debatable
- Use Lock if you need "advanced" properties:
 - Fairness (remember, scheduling is counter-intuitive)
 - Non-structured locking (remember, exceptions are hard)
 - Polling via tryLock (remember, race conditions are tricky)

Signalling. The story is absolutely the same.

Built-in monitors are "friendlier" yet less flexible.

Signalling: broadcast

- Lock+Condition and Monitor provide notifyAll



Question time

Question: Does `notifyAll` or `signalAll` actually force all waiting threads to wake-up?



Signalling: broadcast

- Lock+Condition and Monitor provide `notifyAll`
- Effectively threads are "released" one-by-one, since they are forced to enter critical section.

Signalling: broadcast

- Lock+Condition and Monitor provide `notifyAll`
- Effectively threads are "released" one-by-one, since they are forced to enter critical section.
- Seems that encourages excessive context switch

Thundering herd problem

Scenario:

- many threads wake-up because of broadcast
- only few (e.g. single one) make progress

Thundering herd problem

Scenario:

- many threads wake-up because of broadcast
- only few (e.g. single one) make progress

Problem: inefficient utilization of scheduling quantum

Thundering herd problem

Scenario:

- many threads wake-up because of broadcast
- only few (e.g. single one) make progress

Problem: inefficient utilization of scheduling quantum

Solution:

- Use broadcasting with care

Thundering herd problem

Scenario:

- many threads wake-up because of broadcast
- only few (e.g. single one) make progress

Problem: inefficient utilization of scheduling quanta

Solution:

- Use broadcasting with care
- `monitor.notifyAll` and `condition.signalAll` could avoid this by waking threads one-by-one?

Lecture plan

1 Monitor

- API
- Wait Morphing

2 Broadcast

- CountdownLatch
- Thundering herd problem

3 Group-level concurrency

- Semaphore
- ReadWriteLock

4 ThreadLocal

- API
- Replication
- Snapshotting

5 Thread-safe counters

6 Summary

Disclaimer

- Code on the next slides is a **sketch** implementation of Monitor class
- It excessively uses mutexes and polling with timeout to make the code more compact
- It does not support interrupts as built-in Java monitors (see Lecture 5)
- Real world is much more complicated
 - "Compact Java Monitors" by Dice, Kogan. 2021⁷

We just want to understand how "awake other thread" operation could be delayed until mutex is unlocked.

⁷ <https://arxiv.org/pdf/2102.04188>

Wait morphing

```
class Monitor {  
    Thread owner;  
    int count;  
    Set<Thread> enterSet;  
    static Thread me() { return Thread.currentThread(); }  
    synchronized boolean tryEnter() {  
        if (owner == me()) { count++; return true; }  
        if (owner == null) { count = 1; owner = me(); return true; }  
        return false;  
    }  
    void monitorEnter() { ... }  
    void monitorExit() { ... }  
}
```


Wait morphing

```
class Monitor {  
    void monitorEnter() {  
        synchronized (this) { enterSet.add(me()); }  
        while (!tryEnter()) { suspendMe(timeout=100ms); }  
    }  
    void monitorExit() { Thread e = null;  
        synchronized(this) {  
            if (owner != me()) { throw new IllegalMonitorStateException(); }  
            count--;  
            if (count == 0) { enterSet.remove(me()); owner = null;  
                e = selectRandom(enterSet);  
            }  
        }  
        if (e != null) { awake(e); }  
    }  
}
```

Wait morphing

```
class Monitor { Set<Thread> waitSet; Set<Thread> enterSet; Thread owner;
    void wait() {
        if (owner != me()) { throw new IllegalMonitorStateException(); }
        waitSet.add(me());
        while (waitSet.contains(me())) {
            int cnt = fullyUnlock();
            suspendMe(timeout=100ms);
            relock(cnt);
        }
    }
    void notify() {
        if (owner != me()) { throw new IllegalMonitorStateException(); }
        Thread w = removeRandom(waitSet);
        if (w != null) { awake(w); }
    }
}
```

Wait morphing

```
class Monitor { Set<Thread> waitSet; Set<Thread> enterSet; Thread owner;
    void wait() {
        if (owner != me()) { throw new IllegalMonitorStateException(); }
        waitSet.add(me());
        while (waitSet.contains(me())) {
            int cnt = fullyUnlock();
            suspendMe(timeout=100ms);
            relock(cnt);
        }
    }
    void notify() {
        if (owner != me()) { throw new IllegalMonitorStateException(); }
        Thread w = removeRandom(waitSet); // ~~~~~~
        if (w != null) { synchronized(this) { enterSet.add(w); } }
    }
}
```

Question time

Question: wait morphing is cool! But what if my application really needs to awaken many threads simultaneously?



Lecture plan

- 1 Monitor
 - API
 - Wait Morphing
- 2 Broadcast
 - **CountDownLatch**
 - Thundering herd problem
- 3 Group-level concurrency
 - Semaphore
 - ReadWriteLock
- 4 ThreadLocal
 - API
 - Replication
 - Snapshotting
- 5 Thread-safe counters
- 6 Summary

CountDownLatch



CountDownLatch

CountDownLatch⁸

```
CountDownLatch(int count)
void await()      // awaits counter == 0
void countDown()  // counter = max(counter - 1, 0)
long getCount()   // return counter
```

Common usage patterns:

- CountDownLatch(1) acts as a "gate":
 - **closed** (counter == 1)
 - **open** (counter == 0)
- CountDownLatch(N) acts as a "all threads ready" point:
 - **not-yet-ready** (counter > 0)
 - **ready** (counter == 0)

⁸<https://docs.oracle.com/en/java/javase/11/docs/api/java.base/java/util/concurrent/CountDownLatch.html>

CountDownLatch

Example

```
static CountDownLatch ready = new CountDownLatch(N + 1);
static CountDownLatch start = new CountDownLatch(1);
void main() {
    for (i = 0; i < N; i++) new Thread(new Worker(i)).start();
    ready.countDown(); ready.await();
    start.countDown();
}
void worker() {
    prepare();
    ready.countDown();
    start.await();
}
```


NaiveCountDownLatch

```
class NaiveCountDownLatch {  
    private int count;  
    public CountDownLatch(int count) {  
        if (count < 0) throw new IllegalArgumentException();  
        this.count = count;  
    }  
    synchronized void await()    { while (count != 0) { this.wait(); } }  
    synchronized long getCount() { return count; }  
    synchronized void countDown() {  
        if (count == 1) { count = 0; this.notifyAll(); }  
        else            { count--; }  
    }  
}
```

Homework: efficient CountDownLatch

<https://github.com/Svazars/parallel-programming/tree/main/hw/block1/4.3>

Homework

Task 4.3

- *Implement `CountDownLatch` using `ReentrantLock`/`Condition`/built-in monitors*
- *Ensure it releases threads simultaneously (avoid wait morphing side-effects)*
- *Use condition-per-waiting-thread pattern*

Lecture plan

- 1 Monitor
 - API
 - Wait Morphing
- 2 Broadcast
 - CountdownLatch
 - **Thundering herd problem**
- 3 Group-level concurrency
 - Semaphore
 - ReadWriteLock
- 4 ThreadLocal
 - API
 - Replication
 - Snapshotting
- 5 Thread-safe counters
- 6 Summary

Thundering herd problem

```
static Deque<Object> data = ...
static CountDownLatch dataReady = new CountDownLatch(1);
void main(Collection<Object> workItems) {
    synchronized(data) { data.addAll(workItems); }
    dataReady.countDown();
}
void worker() {
    dataReady.await();
    Object element;
    synchronized(data) { element = data.poll(); }
    process(element);
}
```

Thundering herd problem

Scenario:

- many threads wake-up because of broadcast
- only few (e.g. single one) make progress

Problem: inefficient utilization of scheduling quanta.

Solution:

- Use broadcasting with care

Question time

Question: I am confused, could I observe thundering herd problem when using monitors or could not I?



Lecture plan

- 1 Monitor
 - API
 - Wait Morphing
- 2 Broadcast
 - CountdownLatch
 - Thundering herd problem
- 3 Group-level concurrency
 - Semaphore
 - ReadWriteLock
- 4 ThreadLocal
 - API
 - Replication
 - Snapshotting
- 5 Thread-safe counters
- 6 Summary

Group level concurrency

- Lock/Monitor divides threads into "owner" and "others".
- Condition divides threads into "waiters" and "others".
- CountdownLatch helps to coordinate start/stop for group of threads.

Could we do more?

Group level concurrency

- Lock/Monitor divides threads into "owner" and "others".
- Condition divides threads into "waiters" and "others".
- CountdownLatch helps to coordinate start/stop for group of threads.

Could we do more?

- No more than N threads in critical section

Group level concurrency

- Lock/Monitor divides threads into "owner" and "others".
- Condition divides threads into "waiters" and "others".
- CountdownLatch helps to coordinate start/stop for group of threads.

Could we do more?

- No more than N threads in critical section (semaphore)

Group level concurrency

- Lock/Monitor divides threads into "owner" and "others".
- Condition divides threads into "waiters" and "others".
- CountdownLatch helps to coordinate start/stop for group of threads.

Could we do more?

- No more than N threads in critical section (semaphore)
- One exclusive writer and N independent readers

Group level concurrency

- Lock/Monitor divides threads into "owner" and "others".
- Condition divides threads into "waiters" and "others".
- CountdownLatch helps to coordinate start/stop for group of threads.

Could we do more?

- No more than N threads in critical section (semaphore)
- One exclusive writer and N independent readers (read-write lock)

Lecture plan

- 1 Monitor
 - API
 - Wait Morphing
- 2 Broadcast
 - CountdownLatch
 - Thundering herd problem
- 3 Group-level concurrency
 - Semaphore
 - ReadWriteLock
- 4 ThreadLocal
 - API
 - Replication
 - Snapshotting
- 5 Thread-safe counters
- 6 Summary

Semaphore

Semaphore⁹

```
Semaphore(int permits)  
void acquire() // acquire permit, blocks until available  
void release() // release permit
```

Important notes:

- Perfect to
 - limit number of active threads (parallelism control)
 - bound shared resource usage (concurrent access control)
- No ownership – any thread could release permits

⁹

<https://docs.oracle.com/en/java/javase/11/docs/api/java.base/java/util/concurrent/Semaphore.html>

Question time

Question: Semaphore could limit number of simultaneously executing threads to $N == 1$.
Have we already seen similar behaviour?



Question time

Question: Semaphore could limit number of simultaneously executing threads to $N > 1$.
Have we already seen similar behaviour?



BinaryLock

```
class BinaryLock { // NonReentrantLock
    Semaphore semaphore = new Semaphore(1);
    Thread owner = null;
    void lock() {
        semaphore.acquire();
        owner = Thread.currentThread();
    }
    void unlock() {
        if (owner != Thread.currentThread()) {
            throw new IllegalMonitorStateException();
        }
        owner = null;
        semaphore.release();
    }
}
```

Resource control

```
static Semaphore sema = new Semaphore(N);
static Set<Object> items = ...;
void useItem() {
    sema.acquire();
    try {
        synchronized(items) {
            Object item = selectRandom(items);
            items.remove(item);
        }
        use(item);
        synchronized(items) { items.add(item); }
    } finally { sema.release(); }
}
```

Your own railroad

<https://github.com/Svazars/parallel-programming/tree/main/hw/block1/4.4>

Homework

Task 4.4

- *Implement Semaphore using ReentrantLock/Condition/built-in monitors*

Lecture plan

- 1 Monitor
 - API
 - Wait Morphing
- 2 Broadcast
 - CountdownLatch
 - Thundering herd problem
- 3 Group-level concurrency
 - Semaphore
 - ReadWriteLock
- 4 ThreadLocal
 - API
 - Replication
 - Snapshotting
- 5 Thread-safe counters
- 6 Summary

ReadWriteLock

ReadWriteLock¹⁰

```
Lock readLock();
```

```
Lock writeLock();
```

Read-mostly data structures:

- Concurrent readers, Exclusive writer

Design decisions:

¹⁰

<https://docs.oracle.com/en/java/javase/11/docs/api/java.base/java/util/concurrent/locks/ReadWriteLock.html>

Question time

Question: What are the main design decisions for any mutual exclusion primitive?



ReadWriteLock

ReadWriteLock¹¹

Lock `readLock()`

Lock `writeLock()`

Divides threads into two groups:

- "Simultaneous readers" (many)
- "Exclusive writer" (only one)

Design decisions:

- Reentrancy for writers, reentrancy for readers
- Reader preference vs. Writer preference
- Lock upgrade (promotion): `readLock` -> `writeLock`
- Lock downgrade: `writeLock` -> `readLock`

¹¹<https://docs.oracle.com/en/java/javase/11/docs/api/java.base/java/util/concurrent/locks/ReadWriteLock.html>

ReadWriteLock

Example

```
static Map<K, V> kv = ...;
static ReadWriteLock rw = ...;
V get(K k) {
    rw.readLock().lock();
    try {
        return kv.get(k);
    } finally { rw.readLock().unlock(); }
}
void put(K k, V v) {
    rw.writeLock().lock();
    try {
        kv.put(k, v);
    } finally { rw.writeLock().unlock(); }
}
```


Question time

Question: suppose I have method `foo(int x)` and cannot change the signature. How could I pass additional parameter to the method?



Question time

Question: suppose I have method `foo(int x)` and cannot change the signature. How could I pass additional parameter to the method **in concurrent world**?



Lecture plan

- 1 Monitor
 - API
 - Wait Morphing
- 2 Broadcast
 - CountdownLatch
 - Thundering herd problem
- 3 Group-level concurrency
 - Semaphore
 - ReadWriteLock
- 4 ThreadLocal**
 - API
 - Replication
 - Snapshotting
- 5 Thread-safe counters
- 6 Summary

ThreadLocal

ThreadLocal¹²

```
class ThreadId { static long nextId = 0; static Object guard = new Object();
    static final ThreadLocal<Integer> threadId = new ThreadLocal<Integer>() {
        @Override protected Integer initialValue() {
            synchronized(guard) { return nextId++; };
        }
    };
    public static long get() { return threadId.get(); }
}

void main() {
    long myId = ThreadId.get();
    threadFactory.newThread(() -> { long hisId = ThreadId.get(); });
}
```

¹²<https://docs.oracle.com/en/java/javase/11/docs/api/java.base/java/lang/ThreadLocal.html>

Lecture plan

- 1 Monitor
 - API
 - Wait Morphing
- 2 Broadcast
 - CountdownLatch
 - Thundering herd problem
- 3 Group-level concurrency
 - Semaphore
 - ReadWriteLock
- 4 ThreadLocal
 - API
 - Replication
 - Snapshotting
- 5 Thread-safe counters
- 6 Summary

Prime numbers: pure function

```
class PureMath {  
    static boolean checkPrime(long num) {  
        if (num < 2) { throw new IllegalArgumentException(); }  
        if (num <= 3) { return true; }  
        for (long t = 2; t < sqrt(num) + 1; t++) {  
            if (num % t == 0) { return false; }  
        }  
        return true;  
    }  
}
```

Prime numbers: caching

```
class CachingMath {  
    static Set<Long> knownPrimes = new Set<>();  
    static boolean isPrime(long num) {  
        if (knownPrimes.contains(num)) {  
            return true;  
        } else {  
            boolean prime = checkPrime(num);  
            if (prime) { knownPrimes.add(num); }  
            return prime;  
        }  
    }  
}
```

Prime numbers: concurrent caching

```
class ThreadSafeMath {  
    static Set<Long> knownPrimes = new Set<>();  
    static ReadWriteLock rw = new ReentrantReadWriteLock();  
    static boolean isPrime(long num) {  
        rw.readLock().lock();  
        try    { if (knownPrimes.contains(num)) { return true; } }  
        finally { rw.readLock().unlock(); }  
        boolean isPrime = checkPrime(num);  
        if (isPrime) {  
            rw.writeLock().lock();  
            try    { knownPrimes.add(num); }  
            finally { rw.writeLock().unlock(); }  
        }  
    }  
}
```


Prime numbers: replication

```
class ReplicatedMath {
    static final ThreadLocal<Set<Long>> myPrimes = new ThreadLocal<>() {
        @Override protected Set<Long> initialValue() { return new HashSet<>(); }
    };

    static boolean isPrime(long num) {
        if (myPrimes.get().contains(num)) { return true; }
        rw.readLock().lock();
        try { myPrimes.addAll(knownPrimes); }
        finally { rw.readLock().unlock(); }
        if (myPrimes.get().contains(num)) { return true; }
        boolean prime = checkPrime(num);
        if (prime) { updateKnownPrimes(num); }
        return prime;
    }
}
```

Replication

Idea:

- create thread-local copies of shared data
- prefer using "local cache" without any synchronization
- in-thread operations would be consistent with each other

Potential problems:

- Cross-thread operations could be partially inconsistent
 - global state "lags" (thread-local copy is more informative)
 - thread-private states "lag" (this thread misses info from other threads)

Question time

Question: `myPrimes.addAll(knownPrimes)` executed under read lock. Scalable?



Lecture plan

- 1 Monitor
 - API
 - Wait Morphing
- 2 Broadcast
 - CountdownLatch
 - Thundering herd problem
- 3 Group-level concurrency
 - Semaphore
 - ReadWriteLock
- 4 ThreadLocal
 - API
 - Replication
 - Snapshotting
- 5 Thread-safe counters
- 6 Summary

Snapshotting

```
class PrimeSetSnapshot {  
    private final Set<Long> primes;  
    public final long version;  
    PrimeSetSnapshot(Set<Long> p, long v) {  
        this.primes = Collections.unmodifiableSet(new HashSet<>(p));  
        this.version = v;  
    }  
    static PrimeSetSnapshot initial() {  
        return new PrimeSetSnapshot(Collections.emptySet(), 0);  
    }  
    boolean contains(long num) { return primes.contains(num); }  
    PrimeSetSnapshot with(long num) {  
        return new PrimeSetSnapshot(new HashSet<>(primes).add(num), version + 1);  
    }  
}
```

Snapshotting

```
class SnapshottingMath {  
    private PrimeSetSnapshot latest = PrimeSetSnapshot.initial();  
    private ReadWriteLock rw = new ReentrantReadWriteLock();  
    static final ThreadLocal<PrimeSetSnapshot> myPrimes = new ThreadLocal<>() {  
        @Override protected Set<PrimeSetSnapshot> initialValue() {  
            return PrimeSetSnapshot.initial();  
        }  
    };  
}
```

Snapshotting

```
class SnapshottingMath {  
    PrimeSetSnapshot latest; ThreadLocal<PrimeSetSnapshot> myPrimes;  
    boolean isPrime(long num) {  
        ???  
    }  
}
```

Snapshotting

```
class SnapshottingMath {
    PrimeSetSnapshot latest; ThreadLocal<PrimeSetSnapshot> myPrimes;
    boolean isPrime(long num) {
        PrimeSetSnapshot priv = myPrimes.get(); PrimeSetSnapshot publ = null;
        if (priv.contains(num)) { return true; }
        rw.readLock().lock(); try { publ = latest; } finally { rw.readLock().unlock(); }
        if (priv.version != publ.version) {
            myPrimes.set(publ); return isPrime(num);
        } else {
            boolean prime = checkPrime(num);
            if (prime) { updateLatestSnapshot(priv.with(num)); }
            return prime;
        }
    }
}
```


Snapshotting

Idea:

- use versioning for shared data: immutable data structure + version number
- use immutable snapshot for speculative computation
- if your snapshot is up-to-date – "commit" computation result
- retry otherwise

Potential problems:

- Missing update: shared data changed but version number was not increased
- Starvation: multiple retries
- Memory consumption: too many heavyweight copies
- Internal mutability: speculative computation has side effects

Further reading: seqlock, ABA problem

But they'll never take our FREEDOM! Deadlock-freedom

<https://github.com/Svazars/parallel-programming/tree/main/hw/block1/4.5>

Homework

Task 4.5

- *Deadlock empire. Every. Single. Level.*

Lecture plan

- 1 Monitor
 - API
 - Wait Morphing
- 2 Broadcast
 - CountdownLatch
 - Thundering herd problem
- 3 Group-level concurrency
 - Semaphore
 - ReadWriteLock
- 4 ThreadLocal
 - API
 - Replication
 - Snapshotting
- 5 Thread-safe counters**
- 6 Summary

Synchronized counter

```
final class SynchronizedCounter {  
    private long data;  
    public SynchronizedCounter(long initial) {  
        data = initial;  
    }  
    public synchronized long get() {  
        return data;  
    }  
    public synchronized void add(long delta) {  
        data += delta;  
    }  
}
```

Read-mostly counter

```
final class ReadMostlyCounter {  
    private long data;  
    private ReadWriteLock rw = new ReentrantReadWriteLock();  
    public ReadMostlyCounter(long initial) { data = initial; }  
    public long get() {  
        rw.readLock().lock();  
        try { return data; }  
        finally { rw.readLock().unlock(); }  
    }  
    public void add(long delta) {  
        rw.writeLock().lock();  
        try { data += delta; }  
        finally { rw.writeLock().unlock(); }  
    }  
}
```

Write-mostly counter: lock splitting

```
final class SynchronizedCounter {  
    private long data;  
    public SynchronizedCounter(long initial) {  
        data = initial;  
    }  
    public synchronized long get() {  
        return data;  
    }  
    public synchronized void add(long delta) { // <----- lock splitting  
        data += delta;  
    }  
}
```

Reminder: Homework Task 2.4 (Medium)

Write-mostly counter: distributed counter

Distributed counter:

- N independent long slots – 1 slot per thread
- Every user of distributed counter should register itself
- Thread adds to its own slot (no contention)
- Thread gets from all slots (could take time)

Question time

Question: "1 slot per thread". Which Java class should I use?



Question time

Question: "user of distributed counter should register itself". Should it be reenterable operation?



Question time

Question:

- Thread adds to its own slot (no contention)
- Thread gets from all slots (could take time)

Do you know data structure that divides threads into two groups: "many simultaneous" and "only one"?



Distributed counter: thread-private slots

- N independent long slots – 1 slot per thread

```
class DistributedCounter {  
    class Slot { long data; }  
    static final ThreadLocal<Slot> mySlot = new ThreadLocal<>() {  
        protected Set<PrimeSetSnapshot> initialValue() { return null; }  
    };  
    boolean thisThreadIsRegistered() {  
        return mySlot.get() != null;  
    }  
    void ensureRegistered() {  
        if (!thisThreadIsRegistered()) { mySlot.set(new Slot()); }  
    }  
}
```

Distributed counter: thread-private slots

- N independent long slots – 1 slot per thread

```
class DistributedCounter {  
    class Slot { long data; }  
    static final ThreadLocal<Slot> mySlot = new ThreadLocal<>() {  
        protected Set<PrimeSetSnapshot> initialValue() { return null; }  
    };  
    boolean thisThreadIsRegistered() {  
        return mySlot.get() != null;  
    }  
    void ensureRegistered() {  
        if (!thisThreadIsRegistered()) { mySlot.set(new Slot()); }  
    }  
}
```

- Thread-safe?

Distributed counter: thread-specific lazy initialization

- Every user of distributed counter should register itself

```
class DistributedCounter {  
    ...  
    void add(long delta) {  
        ensureRegistered();  
        ???  
        ???  
        ???  
    }  
}
```

Distributed counter: scalable add

- Thread adds to its own slot (no contention)

```
class DistributedCounter {  
    final ReadWriteLock rw = new ReentrantReadWriteLock();  
    void add(long delta) {  
        ensureRegistered();  
        ???  
        ???  
        ???  
    }  
}
```

Distributed counter: scalable add

- Thread adds to its own slot (no contention)

```
class DistributedCounter {  
    final ReadWriteLock rw = new ReentrantReadWriteLock();  
    void add(long delta) {  
        ensureRegistered();  
        rw.readLock().lock();  
        try { mySlot.get().data++;}  
        finally { rw.readLock().unlock(); }  
    }  
}
```

Distributed counter: consistent get

- Thread gets from all slots (could take time)

```
class DistributedCounter {  
    final ReadWriteLock rw = new ReentrantReadWriteLock();  
    long get() {  
        ???  
        ???  
        ???  
    }  
}
```


Distributed counter: consistent get

- Thread gets from all slots (could take time)

```
class DistributedCounter {  
    final ReadWriteLock rw = new ReentrantReadWriteLock();  
    long get() { rw.writeLock().lock();  
                try { ??? }  
                finally { rw.writeLock().unlock(); }  
            }  
}
```

Distributed counter: consistent get

- Thread gets from all slots (could take time)

```
class DistributedCounter {
    final ReadWriteLock rw = new ReentrantReadWriteLock();
    long get() { rw.writeLock().lock();
                try { ??? }
                finally { rw.writeLock().unlock(); }
            }
    final List<Slot> lst = new ArrayList<>();
    void ensureRegistered() {
        if (thisThreadIsRegistered()) { return; }
        mySlot.set(new Slot());
        rw.writeLock().lock();
        try { lst.add(mySlot.get()); } finally { rw.writeLock().unlock(); }
    }
}
```

Distributed counter: consistent get

- Thread gets from all slots (could take time)

```
class DistributedCounter {
    final ReadWriteLock rw = new ReentrantReadWriteLock();
    long get() { rw.writeLock().lock(); long result = 0;
        try { for (var slot: lst) { result += slot.data; } }
        finally { rw.writeLock().unlock(); }
        return result; }
    final List<Slot> lst = new ArrayList<>();
    void ensureRegistered() {
        if (thisThreadIsRegistered()) { return; }
        mySlot.set(new Slot());
        rw.writeLock().lock();
        try { lst.add(mySlot.get()); } finally { rw.writeLock().unlock(); }
    }
}
```

Distributed counter: takeaways

- Read-mostly and write-mostly designs could be quite different
- `ThreadLocal` helps to organize uncontended execution
- `ReadWriteLock` splits threads into two groups
 - Not necessarily readers and writers!

Distributed counter: takeaways

- Read-mostly and write-mostly designs could be quite different
- `ThreadLocal` helps to organize uncontended execution
- `ReadWriteLock` splits threads into two groups
 - Not necessarily readers and writers!

Teaser

Distributed counter: takeaways

- Read-mostly and write-mostly designs could be quite different
- `ThreadLocal` helps to organize uncontended execution
- `ReadWriteLock` splits threads into two groups
 - Not necessarily readers and writers!

Teaser

- Replication + Snapshotting + Distributed systems = Headache

Distributed counter: takeaways

- Read-mostly and write-mostly designs could be quite different
- `ThreadLocal` helps to organize uncontended execution
- `ReadWriteLock` splits threads into two groups
 - Not necessarily readers and writers!

Teaser

- Replication + Snapshotting + Distributed systems = Headache
 - Wait-free snapshots (Lecture 8)
 - ABA problem (Lecture 9)
 - Replication on CPU level (Lecture 10)

Summary

Mutual exclusion and signalling combined into single universal primitive:

- Monitor
- Wait morphing – scheduler-aware optimization

Java language has special support for monitor-based thread coordination:

- `synchronized` block
- every object has built-in monitor
- any method could be marked as `synchronized`

Bulk thread notification:

- `CountDownLatch`
- Thundering herd problem

Tools for fine-grained group-level thread control:

- Semaphore, `ReadWriteLock`

Thread-specific data could be used for caching:

- replication, snapshotting

Summary: homework

Lecture 4, Tasks 4.*: <https://github.com/Svazars/parallel-programming/blob/main/hw/block1>